UCS420: Cognitive Computing
Assignment 6

Q1. Sensor Readings

Given temperature readings and an average measurement error of +2°C.

```
a. corrected = temperature + 2
Output: [27 30 33 37 40 29 35 42]

b. fahrenheit = (9/5) * corrected + 32
Output: [80.6 86.0 91.4 98.6 104.0 84.2 95.0 107.6]

c. corrected[corrected > 32]
Output: [33 37 40 35 42]

d. Number of readings exceeding 32°C = 5

e. Vectorization applies an operation to an entire NumPy array at once,
without explicitly writing a loop. Boolean indexing creates a condition
for each element and directly selects the values that satisfy it.
This makes array processing concise and generally more efficient.
```

Q2. Daily Steps

```
a. Total steps = 96,600
b. Mean steps = 8,050.00
c. Maximum = 9,500
   Minimum = 4,500
d. Total for each day (axis=0) = [26500 27300 30400]
e. Total for each user (axis=1) = [18300 24500 14400 27000]
f. np.argmax(steps) = 11
   Position = (3, 2)
   Therefore, the maximum is 9,500 at User 4, Day 3.
```

Q3. NumPy Array, Slicing and Reshaping

```
a. original = np.array([1, 2, 3, 4, 5, 6])

b. subset = original[1:5]

c. subset[0] = 999
   original = [1 999 3 4 5 6]
   subset   = [999 3 4 5]
   Explanation: a normal slice is a view, so both refer to the same data.

d. copied = original[1:5].copy()
   copied[0] = 500
   original = [1 999 3 4 5 6]
   copied   = [500 3 4 5]
   Explanation: copy() creates an independent array.

e. np.arange(1, 13).reshape(3, 4)
   [[ 1  2  3  4]
    [ 5  6  7  8]
    [ 9 10 11 12]]

f. First row:  [1 2 3 4]
   Last row:   [9 10 11 12]
   Second col:  [2 6 10]
   Rows 1–2 and columns 2–3:
   [[ 6  7]
    [10 11]]

g. flatten() and ravel() both give:
   [1 2 3 4 5 6 7 8 9 10 11 12]

h. ravel() returns a view when possible, so changing its elements
   can affect the original array.
```

i. flatten() returns a copy, so changing it does not affect original.

j. shape = (3, 4)
   ndim = 2
   size = 12
   dtype = integer (exact integer dtype can vary by system).

Q4. Cognitive-Assistive System

a. X.shape = (4, 3)
   X.ndim = 2

b. X.T =
   [[6 5 8 4]
    [70 50 80 30]
    [3 6 2 8]]

   Transpose changes rows into columns and columns into rows.

c. X.T @ X =
   [[141 1140   92]
    [1140 16500 840]
    [92   840 113]]

d. np.linalg.inv(X.T @ X) gives the inverse of this matrix.

e. OLS:
   beta = np.linalg.inv(X.T @ X) @ X.T @ y

   beta ≈ [-1.217391, 0.326087, 8.608696]

f. The three coefficients correspond to sleep, activity and stress.
   Each coefficient indicates the estimated change in assistance score
   for a one-unit increase in that feature, with the other features
   held constant.

g. new_user = np.array([5, 40, 7])
   predicted_score = new_user @ beta
   Predicted assistance score ≈ 72.83